

Epics (MVP)

1. **Foundation / Repo / CI** — Establish monorepo, baselines, tooling, Dockerized local env, and CI to build/test on PR and publish images on main.
 2. **Auth & Users** — Provide secure email/password authentication, JWT issuance/validation, and basic RBAC (Guest/Host/Admin).
 3. **Listings & Units** — CRUD for listings and units with validation, search fields, and sensible DB indexes.
 4. **Availability & Holds** — Calendar per unit and time-boxed holds with TTL and automatic expiry.
 5. **Booking Flow** — Confirm booking with overlap checks and a mock payment provider.
 6. **Admin Minimal** — Lightweight admin pages to approve listings and view bookings.
 7. **Frontend SPA** — React + TypeScript app with routing, auth, listings UI, search, and empty states.
 8. **Testing & Quality** — Test harnesses (unit/integration/e2e), coverage targets, linting/formatting.
 9. **Observability Basics** — Structured logging, global error handling (Problem Details), health checks, basic request metrics.
 10. **Seed & Fixtures** — Seed data and fixtures to demo flows end-to-end.
-

Sprint 1 (2 weeks)

Focus: repo/infra, DB schema & migrations, auth skeleton, Listings CRUD + search, SPA scaffolding, CI, seeds, logging/error handling.

1) Initialize monorepo & standards

- **Summary:** Initialize monorepo with `/backend`, `/frontend`, `/infra` and shared conventions
- **Issue Type:** Task
- **Epic Link:** Foundation / Repo / CI
- **Description:** Create repo structure with README, contribution guidelines, editorconfig, and CODEOWNERS; enable C# nullable, analyzers, and TS strict.
- **Acceptance Criteria (Gherkin):**
 - Given the repository is cloned, When I open it, Then I see `/backend`, `/frontend`, `/infra` folders and a root README with setup steps.
- **Definition of Done:** Code merged; README updated; linters format on commit; CI recognizes folders.
- **Dependencies:** none
- **Estimate:** 1
- **Labels:** mvp infra docs

2) Docker Compose: Postgres + app stubs

- **Issue Type:** Task
- **Epic Link:** Foundation / Repo / CI
- **Description:** Compose with Postgres 16, pgAdmin optional, backend & frontend services; volumes; healthchecks.

- **AC:**
- Given `docker compose up`, Then Postgres is reachable and backend/frontend containers start.
- **DoD:** Compose file, `.env.sample`, README run steps.
- **Dependencies:** Initialize monorepo & standards
- **Estimate:** 2
- **Labels:** mvp infra db

3) ADR-001: Monorepo & MVP stack

- **Issue Type:** Task
- **Epic Link:** Foundation / Repo / CI
- **Description:** Record decision on monorepo, tech, test strategy, CI choice (GitHub Actions), and coding standards.
- **AC:**
- Given `/docs/adr/0001-monorepo.md`, Then it states alternatives, decision, and consequences.
- **DoD:** ADR committed and linked from README.
- **Dependencies:** Initialize monorepo & standards
- **Estimate:** 1
- **Labels:** mvp docs architecture

4) EF Core: Initial MVP schema & migrations

- **Issue Type:** Task
- **Epic Link:** Foundation / Repo / CI
- **Description:** Implement EF Core models & migrations for Users/Roles, Listings, Units; create DbContexts and migration pipeline.
- **AC:**
- Given migrations are applied, Then tables for User, Role, Listing, Unit exist.
- **DoD:** Migration applied via `dotnet ef database update`; rollback tested locally.
- **Dependencies:** Docker Compose: Postgres + app stubs
- **Estimate:** 2
- **Labels:** mvp backend db

5) GitHub Actions: build, test, docker image

- **Issue Type:** Task
- **Epic Link:** Foundation / Repo / CI
- **Description:** CI on PR (build + tests), on main (build + tests + docker build & push to registry).
- **AC:**
- Given a PR, Then CI runs and reports status.
- Given a merge to main, Then a versioned image is pushed.
- **DoD:** Workflow YAMLS in repo; badges in README.
- **Dependencies:** Initialize monorepo & standards
- **Estimate:** 3
- **Labels:** mvp ci infra

6) User & Role tables + RBAC scaffolding

- **Issue Type:** Task
- **Epic Link:** Auth & Users
- **Description:** Create User (id,email,passwordHash,role) and Role tables; seed Admin.
- **AC:**
 - Given migration runs, Then an Admin user exists with a random secure password printed once in logs.
- **DoD:** Entities, migration, seed, doc on rotating admin password.
- **Dependencies:** EF Core: Initial MVP schema & migrations
- **Estimate:** 1
- **Labels:** mvp backend db auth

7) JWT auth: register/login endpoints

- **Issue Type:** Story
- **Epic Link:** Auth & Users
- **Description:** Implement email/password register and login; hash with PBKDF2/BCrypt; return JWT with role claims.
- **AC:**
 - Given valid email/password, When POST /auth/login, Then 200 with JWT.
 - Given duplicate email, When POST /auth/register, Then 409.
- **DoD:** Controllers, DTOs, validation, unit tests; Swagger examples.
- **Dependencies:** User & Role tables + RBAC scaffolding
- **Estimate:** 2
- **Labels:** mvp backend api auth

8) Minimal authorization middleware (Guest/Host/Admin)

- **Issue Type:** Task
- **Epic Link:** Auth & Users
- **Description:** Attribute-based authorization; protect listing creation to Host/Admin; Admin endpoints restricted.
- **AC:**
 - Given a Guest token, When calling POST /listings, Then 403.
- **DoD:** Policies, tests.
- **Dependencies:** JWT auth: register/login endpoints
- **Estimate:** 1
- **Labels:** mvp backend auth

9) Listing & Unit domain + repository

- **Issue Type:** Task
- **Epic Link:** Listings & Units
- **Description:** Implement entities, EF configurations, repositories with basic CRUD.
- **AC:**
 - Given repository tests, Then CRUD passes and constraints enforced.
- **DoD:** Entities, mappings, tests.

- **Dependencies:** EF Core: Initial MVP schema & migrations
- **Estimate:** 1
- **Labels:** mvp backend db

10) Listings CRUD REST endpoints

- **Issue Type:** Story
- **Epic Link:** Listings & Units
- **Description:** Create, read, update, delete for Listing and Unit; photoUrls are strings (URLs only).
- **AC:**
 - Given valid payload, When POST /listings, Then 201 with id.
 - Given invalid photoUrls, Then 422 with ProblemDetails.
- **DoD:** Endpoints, DTOs, validation rules; integration tests.
- **Dependencies:** Listing & Unit domain + repository | Minimal authorization middleware (Guest/Host/Admin)
- **Estimate:** 3
- **Labels:** mvp backend api

11) Search API: location/dates/guests/type

- **Issue Type:** Story
- **Epic Link:** Listings & Units
- **Description:** Implement GET /search with filters; sort by relevance; add DB indexes for city, type, guests.
- **AC:**
 - Given city and guests, Then results only include listings meeting capacity.
 - Given dates, Then only listings with all units open are included.
- **DoD:** Endpoint, index migration, integration tests.
- **Dependencies:** Listing & Unit domain + repository
- **Estimate:** 3
- **Labels:** mvp backend api db

12) React scaffold: Vite, routing, UI kit

- **Issue Type:** Task
- **Epic Link:** Frontend SPA
- **Description:** Scaffold React TS app with routing, layout, tailwind (or minimal CSS), axios client.
- **AC:**
 - Given `npm run dev`, Then the app loads with base layout and routes.
- **DoD:** App skeleton committed; basic smoke tests.
- **Dependencies:** Initialize monorepo & standards
- **Estimate:** 1
- **Labels:** mvp frontend

13) Auth pages: Register/Login + token storage

- **Issue Type:** Story
- **Epic Link:** Frontend SPA

- **Description:** Build forms with validation; store JWT in memory + refresh on reload via localStorage; logout.
- **AC:**
- Given valid credentials, Then user is redirected and sees their role in header.
- Given invalid credentials, Then error toast shown.
- **DoD:** Pages, hooks, tests.
- **Dependencies:** JWT auth: register/login endpoints | React scaffold: Vite, routing, UI kit
- **Estimate:** 2
- **Labels:** mvp frontend auth

14) Listings UI: list/create/edit with photo URLs

- **Issue Type:** Story
- **Epic Link:** Frontend SPA
- **Description:** Pages to view list, create, and edit listings and units.
- **AC:**
- Given Host role, When creating a listing, Then validation prevents empty title and invalid URL.
- Then list view updates without page reload.
- **DoD:** Pages, forms, API integration, RTL tests.
- **Dependencies:** Listings CRUD REST endpoints | React scaffold: Vite, routing, UI kit
- **Estimate:** 3
- **Labels:** mvp frontend listings

15) Empty states & error toasts

- **Issue Type:** Task
- **Epic Link:** Frontend SPA
- **Description:** Show empty states for no results; generic error handling hooked to ProblemDetails.
- **AC:**
- Given 404/422 responses, Then user sees friendly messages.
- **DoD:** Components, tests.
- **Dependencies:** Listings UI: list/create/edit with photo URLs
- **Estimate:** 1
- **Labels:** mvp frontend ux

16) Observability: Serilog + global error handler + health checks

- **Issue Type:** Story
- **Epic Link:** Observability Basics
- **Description:** Structured logging, centralized exception middleware returning RFC7807 ProblemDetails, `/health` endpoint.
- **AC:**
- Given an unhandled error, Then HTTP 500 with ProblemDetails is returned and event logged.
- Given GET `/health`, Then `Healthy` returned when DB reachable.
- **DoD:** Middleware, filters, tests, docs.
- **Dependencies:** Docker Compose: Postgres + app stubs
- **Estimate:** 2
- **Labels:** mvp backend infra

17) Swagger/OpenAPI + Postman collection

- **Issue Type:** Task
- **Epic Link:** Foundation / Repo / CI
- **Description:** Expose Swagger UI and export collection for Postman/VS Code REST Client.
- **AC:**
 - Given GET /swagger, Then UI is reachable; Postman collection committed under `/docs/api`.
- **DoD:** OpenAPI annotations; CI artifact.
- **Dependencies:** Listings CRUD REST endpoints | JWT auth: register/login endpoints
- **Estimate:** 1
- **Labels:** mvp backend api docs

18) Test harnesses baseline (xUnit, WebAppFactory, Testcontainers)

- **Issue Type:** Task
- **Epic Link:** Testing & Quality
- **Description:** Set up unit and integration test projects; run against ephemeral Postgres via Testcontainers.
- **AC:**
 - Given `dotnet test`, Then unit + integration tests run in CI.
- **DoD:** Sample tests for auth & listings; CI passes.
- **Dependencies:** GitHub Actions: build, test, docker image | EF Core: Initial MVP schema & migrations
- **Estimate:** 2
- **Labels:** mvp backend testing

19) Frontend testing baseline (Jest + RTL)

- **Issue Type:** Task
- **Epic Link:** Testing & Quality
- **Description:** Configure Jest/RTL; add smoke test for listings page.
- **AC:**
 - Given `npm test`, Then unit tests pass in CI.
- **DoD:** Configured scripts; coverage reported.
- **Dependencies:** React scaffold: Vite, routing, UI kit
- **Estimate:** 1
- **Labels:** mvp frontend testing

20) Seed data & fixtures

- **Issue Type:** Task
- **Epic Link:** Seed & Fixtures
- **Description:** Seed two Hosts, one Admin, 10 Listings, 15 Units; fixtures for tests.
- **AC:**
 - Given fresh DB, Then `dotnet run -- seed` loads sample data.
- **DoD:** Seeder, docs, CI smoke step.
- **Dependencies:** EF Core: Initial MVP schema & migrations
- **Estimate:** 2
- **Labels:** mvp backend db testing

21) Sample env variables & secrets docs

- **Issue Type:** Task
 - **Epic Link:** Seed & Fixtures
 - **Description:** Provide `.env.sample` covering DB, JWT secret, ports; document secret management.
 - **AC:**
 - Given `.env.sample`, Then new devs can run locally in <10 minutes.
 - **DoD:** Docs and template committed.
 - **Dependencies:** Docker Compose: Postgres + app stubs
 - **Estimate:** 1
 - **Labels:** mvp infra docs
-

Sprint 2 (2 weeks)

Focus: Availability calendar, holds with TTL+expiry job, booking confirm (mock payment), minimal admin, e2e tests & hardening.

1) Calendar & Hold tables + migrations

- **Issue Type:** Task
- **Epic Link:** Availability & Holds
- **Description:** Create UnitCalendar (unitId,date,status,holdId?) and Hold (id,unitId,startDate,endDate,guestId,expiresAt,status).
- **AC:**
- Given migration runs, Then calendar and hold tables exist with constraints.
- **DoD:** Migrations, EF configs, seed a few blocks.
- **Dependencies:** EF Core: Initial MVP schema & migrations
- **Estimate:** 2
- **Labels:** mvp backend db

2) Availability query endpoint

- **Issue Type:** Story
- **Epic Link:** Availability & Holds
- **Description:** GET /units/{id}/availability?from=..&to=.. returns open/blocked/booked per date.
- **AC:**
- Given a valid range, Then status is accurate; Given invalid range (>90 days), Then 422.
- **DoD:** Endpoint, tests, docs.
- **Dependencies:** Calendar & Hold tables + migrations
- **Estimate:** 2
- **Labels:** mvp backend api

3) Create Hold (atomic validation)

- **Issue Type:** Story
- **Epic Link:** Availability & Holds

- **Description:** POST /holds validates no overlap with existing bookings/holds and creates rows in a serializable transaction; sets `expiresAt`.
- **AC:**
 - Given an overlapping existing hold, Then 409.
 - Given `startDate > endDate`, Then 422.
- **DoD:** Transactional service, tests for race conditions.
- **Dependencies:** Availability query endpoint
- **Estimate:** 3
- **Labels:** mvp backend api db

4) Expire holds background job (Asia/Dubai)

- **Issue Type:** Story
- **Epic Link:** Availability & Holds
- **Description:** Background job runs every minute; sets `status=expired` when `expiresAt < now()` and releases calendar days. Use Asia/Dubai for logs/examples.
- **AC:**
 - Given a hold with `expiresAt=2025-11-01T10:00:00 Asia/Dubai`, When time passes, Then status becomes expired and days open.
- **DoD:** HostedService, metrics log, tests advancing clock.
- **Dependencies:** Create Hold (atomic validation)
- **Estimate:** 2
- **Labels:** mvp backend infra scheduling

5) Booking table + migration

- **Issue Type:** Task
- **Epic Link:** Booking Flow
- **Description:** Create Booking (id,guestId,unitId,startDate,endDate,price,status).
- **AC:**
 - Given migration runs, Then Booking table exists.
- **DoD:** EF entity/config; seed a confirmed booking.
- **Dependencies:** Calendar & Hold tables + migrations
- **Estimate:** 1
- **Labels:** mvp backend db

6) Confirm booking (mock payment)

- **Issue Type:** Story
- **Epic Link:** Booking Flow
- **Description:** POST /bookings/confirm consumes an active hold, simulates payment (mock provider), and writes Booking; prevents overlaps.
- **AC:**
 - Given an expired hold, Then 409 and no booking is created.
 - Given success, Then Hold becomes consumed and calendar dates become booked.
- **DoD:** Endpoint, domain service, tests.
- **Dependencies:** Booking table + migration | Create Hold (atomic validation)
- **Estimate:** 3

- **Labels:** mvp backend api

7) Admin: approve listing & view bookings

- **Issue Type:** Story
- **Epic Link:** Admin Minimal
- **Description:** Admin UI to approve listing and a grid to view bookings with filters.
- **AC:**
 - Given Admin role, Then Approve button is visible and persists status.
- **DoD:** Pages, API calls, e2e tests.
- **Dependencies:** Listings CRUD REST endpoints | Confirm booking (mock payment)
- **Estimate:** 2
- **Labels:** mvp frontend admin

8) Frontend: unit availability calendar UI

- **Issue Type:** Story
- **Epic Link:** Frontend SPA
- **Description:** Calendar component shows open/blocked/booked; select dates to hold; error states.
- **AC:**
 - Given blocked days, Then they are disabled.
- **DoD:** Component, tests, accessibility.
- **Dependencies:** Availability query endpoint
- **Estimate:** 2
- **Labels:** mvp frontend availability

9) Frontend: hold flow + countdown UI

- **Issue Type:** Story
- **Epic Link:** Frontend SPA
- **Description:** Initiate hold from calendar; show countdown to `expiresAt`; allow cancel hold.
- **AC:**
 - Given an active hold, Then countdown updates every second.
- **DoD:** Components, API hooks, tests.
- **Dependencies:** Create Hold (atomic validation) | Frontend: unit availability calendar UI
- **Estimate:** 3
- **Labels:** mvp frontend holds

10) Frontend: booking confirmation UI & receipt

- **Issue Type:** Task
- **Epic Link:** Frontend SPA
- **Description:** Confirmation page with booking summary (dates, price, unit, ref #) and printable receipt.
- **AC:**
 - Given a confirmed booking, Then user can print receipt.
- **DoD:** Page, tests.
- **Dependencies:** Confirm booking (mock payment)
- **Estimate:** 1

- **Labels:** mvp frontend booking

11) Hardening: overlap/edge-case tests

- **Issue Type:** Task
- **Epic Link:** Testing & Quality
- **Description:** Add tests for overlapping holds/bookings, invalid ranges, concurrent holds.
- **AC:**
 - Given concurrent requests on same unit/dates, Then only one hold succeeds.
- **DoD:** Integration tests; CI green.
- **Dependencies:** Create Hold (atomic validation) | Confirm booking (mock payment)
- **Estimate:** 2
- **Labels:** mvp backend testing

12) Playwright e2e: browse→hold→confirm

- **Issue Type:** Story
- **Epic Link:** Testing & Quality
- **Description:** End-to-end scenario from search to booking confirmation.
- **AC:**
 - Given seeded data, Then e2e flow passes in CI headless mode.
- **DoD:** Playwright project, CI job, screenshots on failure.
- **Dependencies:** Frontend: hold flow + countdown UI | Confirm booking (mock payment)
- **Estimate:** 2
- **Labels:** mvp testing e2e

13) Improve ProblemDetails & FE toasts mapping

- **Issue Type:** Task
- **Epic Link:** Observability Basics
- **Description:** Map backend codes to FE toasts; add correlation id to logs and echo in responses.
- **AC:**
 - Given an error, Then response includes correlation id that appears in logs.
- **DoD:** Middleware update, FE mapping, docs.
- **Dependencies:** Observability: Serilog + global error handler + health checks
- **Estimate:** 1
- **Labels:** mvp backend frontend observability

14) README: end-to-end setup & demo script

- **Issue Type:** Task
- **Epic Link:** Seed & Fixtures
- **Description:** Add complete setup steps and a demo script to showcase flows.
- **AC:**
 - Given a new dev, Then they can run the demo in <15 minutes.
- **DoD:** README updated; GIFs optional.
- **Dependencies:** Seed data & fixtures | Playwright e2e: browse→hold→confirm
- **Estimate:** 1
- **Labels:** mvp docs

15) Coverage threshold gates (70% target domain/services)

- **Issue Type:** Task
- **Epic Link:** Testing & Quality
- **Description:** Configure coverage report & threshold; fail CI if below thresholds for backend domain/ services and FE components.
- **AC:**
 - Given CI run, Then coverage % is printed and enforced.
 - **DoD:** Coverage tooling wired; docs note exceptions.
- **Dependencies:** Test harnesses baseline (xUnit, WebAppFactory, Testcontainers) | Frontend testing baseline (Jest + RTL)
- **Estimate:** 1
- **Labels:** mvp testing ci

Jira CSV (importable)

Columns:	Project	Key, Issue	Type, Summary, Description, Epic	Link, Labels, Story
	Points, Dependencies			

```
Project Key,Issue Type,Summary,Description,Epic Link,Labels,Story
Points,Dependencies
{JIRA_PROJECT_KEY},Epic,Foundation / Repo / CI,"Epic Name: Foundation / Repo /
CI
Establish monorepo, Dockerized env, and CI baselines to build/test and publish
images.",,"mvp infra ci docs",,
{JIRA_PROJECT_KEY},Epic,Auth & Users,"Epic Name: Auth & Users
Secure email/password auth with JWT and basic RBAC.",,"mvp backend auth",,
{JIRA_PROJECT_KEY},Epic,Listings & Units,"Epic Name: Listings & Units
CRUD and search for listings and units with validation and indexes.",,"mvp
backend api db",,
{JIRA_PROJECT_KEY},Epic,Availability & Holds,"Epic Name: Availability & Holds
Per-unit calendars and time-boxed holds with TTL and expiry.",,"mvp backend api
db",,
{JIRA_PROJECT_KEY},Epic,Booking Flow,"Epic Name: Booking Flow
Confirm booking via consumed holds and mock payment.",,"mvp backend api",,
{JIRA_PROJECT_KEY},Epic,Admin Minimal,"Epic Name: Admin Minimal
Approve listings and view bookings.",,"mvp frontend admin",,
{JIRA_PROJECT_KEY},Epic,Frontend SPA,"Epic Name: Frontend SPA
React+TS app with routing, auth, listings, search, and UX basics.",,"mvp
frontend",,
{JIRA_PROJECT_KEY},Epic,Testing & Quality,"Epic Name: Testing & Quality
Test harnesses, coverage, lint/format, and e2e.",,"mvp testing ci",,
{JIRA_PROJECT_KEY},Epic,Observability Basics,"Epic Name: Observability Basics
Structured logging, ProblemDetails, health checks, correlation IDs.",,"mvp
```

```

backend observability",,
{JIRA_PROJECT_KEY},Epic,Seed & Fixtures,"Epic Name: Seed & Fixtures
Seed data and fixtures to demo flows and support tests.",,"mvp db testing
docs",,
{JIRA_PROJECT_KEY},Task,Initialize monorepo with /backend /frontend /
infra,"Create repo structure; add README, editorconfig, CODEOWNERS; enable C#
nullable, analyzers, and TS strict.",Foundation / Repo / CI,"mvp infra docs",1,
{JIRA_PROJECT_KEY},Task,Docker Compose for Postgres and app stubs,"Compose
services for Postgres 16, backend, frontend; volumes and
healthchecks; .env.sample and README run steps.",Foundation / Repo / CI,"mvp
infra db",2,Initialize monorepo with /backend /frontend /infra
{JIRA_PROJECT_KEY},Task,ADR-001 Monorepo & MVP stack,"Record decision drivers,
options, decision, consequences; link from README.",Foundation / Repo / CI,"mvp
docs architecture",1,Initialize monorepo with /backend /frontend /infra
{JIRA_PROJECT_KEY},Task,EF Core initial schema & migrations,"Implement EF models
and migrations for User/Role/Listing/Unit; enable migration
pipeline.",Foundation / Repo / CI,"mvp backend db",2,Docker Compose for Postgres
and app stubs
{JIRA_PROJECT_KEY},Task,GitHub Actions build test and image,"PR: build+tests;
main: build+tests+docker push; badges in README.",Foundation / Repo / CI,"mvp ci
infra",3,Initialize monorepo with /backend /frontend /infra
{JIRA_PROJECT_KEY},Task,User & Role tables with Admin seed,"Create tables and
seed Admin user with secure password.",Auth & Users,"mvp backend db auth",1,EF
Core initial schema & migrations
{JIRA_PROJECT_KEY},Story,JWT auth register and login endpoints,"Email/password
register/login; PBKDF2/BCrypt hashing; JWT with role claims.",Auth & Users,"mvp
backend api auth",2,User & Role tables with Admin seed
{JIRA_PROJECT_KEY},Task,Minimal authorization middleware and
policies,"Attribute-based authorization; restrict Host/Admin operations.",Auth &
Users,"mvp backend auth",1,JWT auth register and login endpoints
{JIRA_PROJECT_KEY},Task,Listing & Unit domain and repository,"Entities, EF
configurations, repositories with CRUD; unit tests.",Listings & Units,"mvp
backend db",1,EF Core initial schema & migrations
{JIRA_PROJECT_KEY},Story,Listings CRUD REST endpoints,"Create/read/update/delete
listings and units; validate photoUrls as URLs.",Listings & Units,"mvp backend
api",3,"Listing & Unit domain and repository|Minimal authorization middleware
and policies"
{JIRA_PROJECT_KEY},Story,Search API for location dates guests type,"GET /search
with filters; add indexes for city,type,guests.",Listings & Units,"mvp backend
api db",3,Listing & Unit domain and repository
{JIRA_PROJECT_KEY},Task,React scaffold with Vite routing and UI kit,"Scaffold
React TS app with routing/layout and axios client; smoke tests.",Frontend
SPA,"mvp frontend",1,Initialize monorepo with /backend /frontend /infra
{JIRA_PROJECT_KEY},Story,Auth pages register login and token storage,"Forms with
validation; store JWT; logout.",Frontend SPA,"mvp frontend auth",2,"JWT auth
register and login endpoints|React scaffold with Vite routing and UI kit"
{JIRA_PROJECT_KEY},Story,Listings UI list create edit,"Pages for listing list/
create/edit with validation and live refresh.",Frontend SPA,"mvp frontend

```

```

listings",3,"Listings CRUD REST endpoints|React scaffold with Vite routing and
UI kit"
{JIRA_PROJECT_KEY},Task,Empty states and error toasts,"Friendly empty states and
ProblemDetails-to-toast mapping.",Frontend SPA,"mvp frontend ux",1,Listings UI
list create edit
{JIRA_PROJECT_KEY},Story,Observability global error handler and health
checks,"Serilog structured logging; ProblemDetails middleware; /health
endpoint.",Observability Basics,"mvp backend infra",2,Docker Compose for
Postgres and app stubs
{JIRA_PROJECT_KEY},Task,Swagger OpenAPI and Postman collection,"Expose Swagger
UI; export collection under /docs/api; publish as CI artifact.",Foundation /
Repo / CI,"mvp backend api docs",1,"Listings CRUD REST endpoints|JWT auth
register and login endpoints"
{JIRA_PROJECT_KEY},Task,Backend test harnesses with Testcontainers,"xUnit
unit+integration setup; run ephemeral Postgres; sample tests for auth and
listings.",Testing & Quality,"mvp backend testing",2,"GitHub Actions build test
and image|EF Core initial schema & migrations"
{JIRA_PROJECT_KEY},Task,Frontend testing baseline with Jest RTL,"Configure Jest/
RTL; add smoke test for listings page; report coverage.",Testing & Quality,"mvp
frontend testing",1,React scaffold with Vite routing and UI kit
{JIRA_PROJECT_KEY},Task,Seed data and fixtures,"Seed Hosts/Admin/Listings/Units;
fixtures for tests; CLI seed command.",Seed & Fixtures,"mvp backend db testing",
2,EF Core initial schema & migrations
{JIRA_PROJECT_KEY},Task,Sample .env and secrets docs,"Provide .env.sample and
secret guidance; ports and JWT secret.",Seed & Fixtures,"mvp infra docs",
1,Docker Compose for Postgres and app stubs
{JIRA_PROJECT_KEY},Task,Calendar and Hold tables migrations,"Create UnitCalendar
and Hold tables; EF config and seed blocks.",Availability & Holds,"mvp backend
db",2,EF Core initial schema & migrations
{JIRA_PROJECT_KEY},Story,Availability query endpoint,"GET /units/{id}/
availability returns per-day status; validate date ranges.",Availability &
Holds,"mvp backend api",2,Calendar and Hold tables migrations
{JIRA_PROJECT_KEY},Story,Create hold with atomic validation,"POST /holds
validates overlaps and creates hold in serializable transaction; sets
expiresAt.",Availability & Holds,"mvp backend api db",3,Availability query
endpoint
{JIRA_PROJECT_KEY},Story,Expire holds background job Asia Dubai,"Background job
every minute to expire holds and free days; logs in Asia/Dubai.",Availability &
Holds,"mvp backend infra scheduling",2,Create hold with atomic validation
{JIRA_PROJECT_KEY},Task,Booking table and migration,"Create Booking table with
fields and constraints; seed one booking.",Booking Flow,"mvp backend db",
1,Calendar and Hold tables migrations
{JIRA_PROJECT_KEY},Story,Confirm booking using mock payment,"POST /bookings/
confirm consumes active hold; prevents overlap; writes booking.",Booking
Flow,"mvp backend api",3,"Booking table and migration|Create hold with atomic
validation"
{JIRA_PROJECT_KEY},Story,Admin approve listing and view bookings,"Admin UI to
approve listings and view bookings with filters.",Admin Minimal,"mvp frontend

```

```

admin",2,"Listings CRUD REST endpoints|Confirm booking using mock payment"
{JIRA_PROJECT_KEY},Story,FE unit availability calendar UI,"Calendar component
showing open/blocked/booked; date selection.",Frontend SPA,"mvp frontend
availability",2,Availability query endpoint
{JIRA_PROJECT_KEY},Story,FE hold flow and countdown UI,"Initiate hold from
calendar; show countdown; cancel hold.",Frontend SPA,"mvp frontend holds",
3,"Create hold with atomic validation|FE unit availability calendar UI"
{JIRA_PROJECT_KEY},Task,FE booking confirmation page and receipt,"Display
booking summary and printable receipt after confirmation.",Frontend SPA,"mvp
frontend booking",1,Confirm booking using mock payment
{JIRA_PROJECT_KEY},Task,Edge-case tests overlap and concurrency,"Tests for
overlapping holds/bookings and concurrent requests.",Testing & Quality,"mvp
backend testing",2,"Create hold with atomic validation|Confirm booking using
mock payment"
{JIRA_PROJECT_KEY},Story,Playwright e2e browse hold confirm,"E2E scenario from
search to booking confirmation in CI.",Testing & Quality,"mvp testing e2e",2,"FE
hold flow and countdown UI|Confirm booking using mock payment"
{JIRA_PROJECT_KEY},Task,Improve ProblemDetails and correlation IDs,"Map backend
errors to FE toasts; add correlation id to logs and responses.",Observability
Basics,"mvp backend frontend observability",1,Observability global error handler
and health checks
{JIRA_PROJECT_KEY},Task,README end-to-end setup and demo script,"Document full
setup and a demo script to showcase flows.",Seed & Fixtures,"mvp docs",1,"Seed
data and fixtures|Playwright e2e browse hold confirm"
{JIRA_PROJECT_KEY},Task,Coverage threshold gates 70 percent,"Configure coverage
thresholds for backend domain/services and FE components; fail CI when
below.",Testing & Quality,"mvp testing ci",1,"Backend test harnesses with
Testcontainers|Frontend testing baseline with Jest RTL"

```

Notes & Conventions - All dates/times in examples use **Asia/Dubai**. - MVP keeps images as **URLs** only (no file uploads yet). - Apply **INVEST** and **MoSCoW**: everything here is Must/Should for MVP; later phases are out of Sprint 1. - API: REST + JSON; publish **Swagger/OpenAPI** and a Postman collection. - Testing target: ~70% coverage for domain/services; focus on critical paths (auth, listings, holds, booking).